



Course content – 6 Months Diploma in HPC

C-HUK

March -2025

Sr.No	Module Name & sub topics of those modules	Total Duration
Pre-requisite for HPC		
1	Linux and Operating System Linux Commands, Shell Scripting, Processes, Scheduling & synchronization, Memory management, File Systems, Package Managers (RPM, APM, etc.),	1 week
2	Introduction to HPC & Its Architecture Cluster computer and its architecture, parallel processing concepts, levels of parallelism (task, instruction, thread, memory), parallel programming models (SIMD, MISD, MIMD), super scalar architecture, multicore programming, scheduling in HPC, job partitioning and allocation , synchronization, performance analysis algorithm, profilers, debugging, power management in HPC (power aware processing techniques, s/w power management), peta scale and exascale computing, Overview of GPUs and architecture (AMD and NVIDIA)	1 week
3	Computer Networking & Interconnects • Internetworking • OSI model , TCP/IP • Network protocols • IP Subnetting • IPV4 vs IPV6 • Switches, Router, Interconnect Network (types, GB ethernet, OmniPath Architecture, and Infiniband) • Types of Storage • Components of a disk drive, Physical disk and factors	1 week



Centre for HPC Upskilling & Knowledge-Sharing



	affecting disk drive performance - RAID level performance and availability considerations - Study of various Parallel file systems (GPFS, Lustre File system Quota)	
Programming concepts with Parallel Programming		
4	Software Programming Concepts - C and Data structure - C++ with parallel programming - Python with parallel programming DETAILED CONTENT IN APPENDIX - I	6 weeks
5	Cloud Computing & operation Deployment Strategies - Introduction to cloud computing - Virtualization - Containers - Delivery models in Cloud Computing (IaaS, PaaS, SaaS), Cloud Deployment (Private, Public and Hybrid)	1 week
6	Advanced Programming - Introduction to Accelerator Programming - OpenCL/SYCL - OpenACC (AMD GPU for HIP Programming) - Introduction to CUDA - Tools – Profiling & Debugging, optimizations, Benchmarking, Application check pointing, Performance measurement and optimization	2 weeks
7	Introduction to ML ,DL, and OpenVino - Introduction to Machine Learning - Unsupervised Learning - Clustering - K-Means - Supervised Learning - Classification and Regression: Decision Tree, Naive Bayes, and Logistic Regression - Ensemble Learning - Bagging, Boosting and Stacking - Random Forest, Gradient Boosting, Cat Boost and XGBoost	1 week



	• Deep Learning - Basics of Neural Networks, CNN, RNN, LSTM • Deploy High-Performance, Deep Learning Inference using OpenVino • Deployment of ML/DL models using Flask • ML/DL Frameworks: Scikitlearn, Keras	
8	Statistical analysis and data handling using python (before ML DL) • Basics of Statistics • Probability and Distribution functions • Inferential Statistics • Exploratory Data Analysis (EDA) using NumPy and Pandas • Data Manipulation - Text (Tabular & Webscraping)), Images, Audio and Video • Data Visualization using PowerBI	1 week
10	Aptitude and Business Communication	To be conducted in parallel with other sessions / weekends
11	SWEBOK based Project work	Starts with the programming modules + Additional 3 Weeks

ANNEXURE – I

C with Data Structures

Objective: At the end of the ten day session participants will be very comfortable with programming C at advanced level for systems programming and with applying Advanced Data Structures & Algorithms for industry standard applications.

The session has two inter-leaved threads of topics presented. First thread includes 'C' language topics while the second thread has Data Structures and Algorithms at an advanced level. Data Structures and Algorithm are introduced early in the session to provide sufficient time for implementations.

Prerequisites

The participants are expected to have acquired good practical working knowledge of the prerequisites listed below, prior to attending the session.

Basic Types:

- Integer types: char, short, int, long and the unsigned integers
- Two's complement system
- Floating types, float, double, long double,

Control flow

- if-else-if, for(), while(), do{}while(), switch, pitfalls with goto
- Infinite loops

Arrays

- Single dimensional arrays
- Initialization
- Operations on array elements

Pointers

- Declaration, Initialization,
- Pointers to elements of arrays
- Dynamic memory allocation

Functions

- Call by value and reference

Derived Types

- Structures and Unions
- Initializations
- Operations on members

DAY1:

History of C

C standards

- Non-ANSI C and ANSI C, C89 and ISO C99.

Overview of the language and similarities to assembly

Core language and the C library

Automatic usage of the C library

Calling library functions and including their header files

Data types

Integer types, Unsigned integer types and Floating types
Character types: char, signed char and unsigned char.

Limits for the numerical types as defined in the header files <limits.h>
Optimal numeric types for a given CPU

Constants and Their types

Default types for integer constants
Octal and hexadecimal prefixes
Unsigned, long and long long suffixes
Default types for floating constants
Float and long double suffixes
Hexadecimal floating constants
Character constants
Escape sequences for newline, carriage return, vertical tab, backspace, tab and form feed.
Octal and hexadecimal escape sequences

Identifiers

Definition
Rules and conventions to be followed for naming
Purpose of identifiers

Standard I/O Functions

Buffered I/O by the C library
Line buffered I/O to terminal
Char functions: getc(), putc() and ungetc()
String I/O functions: gets() and puts()
Buffer overflow defect with gets() and using fgets()
Formatted I/O functions: printf() and scanf()
 Displaying pointer values
 Length modifiers for short, long and long long types
 White space separations between values
 Width specifications for integers and floating types
 Precision specification for floating types
 Avoiding buffer overflow with scanf() for string input
 Flag characters: #, 0, -, + and space

memset(), memcpy(), memmove() and memcmp()

Multidimensional arrays

Two dimensional arrays, array of arrays
Initialization
I/O using two dimensional arrays
Implementing matrix addition and multiplication
Extending to three and multi-dimensional arrays

Exercise: - Expression evaluations

Postfix expressions
Conversion from infix notation to post fix notation

DAY2:

Strings

- Strings as an array of chars
- Definition and storage
- String literals (constant strings) and storage
- Using string to initialize char pointers and char arrays
- Revisiting string I/O with gets(), puts(), printf() and scanf().
- Contrasting string I/O with character I/O

- List of strings as a two dimensional array of chars
- String I/O with two dimensional array of chars

- Character processing with
isalpha(), isalnum(), isdigit(), toupper() and tolower()

- String processing with
strlen(), strcpy(), strncpy(), strcmp ()and strncmp()
strchr(), strrchr(), strstr(), strtok(),
atoi() and atof()

- Functions for I/O with string target
- Input from string with sscanf()
- Output to string with sprintf() and snprintf()

Expressions:

Operators:

- Relational
- Arithmetic
- logical
- Assignment
- Increment and Decrement
- Conditional
- Bit wise
- Comparison of logical and bit wise operators
- Special operators
- Coma operator
- Sizeof
- Definition of [] operator
- Type cast operator

- Through study of operator precedence and associativity
- Order of evaluation of expressions: left to right & right to left
- Demonstration of order of evaluation with a simple yacc & lex program
- Order of evaluation for &&,|| and ?: operators
- Side effect operators and implementation dependencies
- Increment, decrement and assignment
- sizeof() operator and non evaluations of operand
- Lvalues, Modifiable Lvalues and Rvalues

Sequence points

- Effects of sequence points
- List of sequence point definitions

Type Conversions

- Implicit type conversion in expressions and promotions
- Explicit type casting
- Conversions with expressions having signed and unsigned operands
- Sign preserving Vs Value preserving conversions

Functions

- Definitions, Invocation and parameter passing
- Concepts of call by value and by reference
- Stack frames
- Buffer over flow vulnerability in functions like gets()
- Function prototypes
- Void return type and void arguments
- main() function specifics, return type
- Command line arguments
- Recursive functions, Tail recursion
- Variable argument list functions
- Defining printf() functions using va_arg() macros
- Library functions for error processing
- errno variable, perror() and strerror() functions

DAY3:**Algorithm Analysis**

- Need for theoretical Asymptotic Analysis
- Inadequacy of practical tests
- Big O, Omega and Theta Notations
- Time complexity
- Space complexity
- Behavior of standard mathematical functions that occur frequently with Big O analysis
- Abstract Data Structure Definition

Linked lists

- Single linked lists
- Doubly linked lists
- Circular linked lists
- Implementation of the above lists

Stacks

- Implementation with arrays and linked lists
- Constant time $O(1)$ operations

Queues

- Implementation with arrays and linked lists
- Constant time $O(1)$ operations

Exercise: - Merging Two Sorted Lists
Stack Packet (Stack with packet size)

DAY4:

Pointers

- Definitions

- Invalid pointers, illegal memory access and segmentation faults

- Pointers to integers & characters

- Pointer initialization

- Pointers into single dimensional integer and character arrays.

- Pointer Arithmetic, addition and subtractions

- Dynamic memory allocation with malloc() family of functions

 - Freeing of dynamic memory

 - Adjusting memory requirements, using realloc()

 - Allocating n elements of a given size

- Pointer to a constant Vs constant pointer

- Arrays and pointers: Similarities and differences

- Void pointers and alignment restrictions for data types

 - Argument types of memset() and memcpy() kind of functions

- Array of pointers

 - Allocating memory for pointers in the array of pointers

- Pointer to array

 - Arithmetic of pointer to an array

 - Allocating memory for pointer to an array

- Double pointers

 - Allocating memory for two dimensional array using double pointers

- Function pointers

 - Definitions and Initializations

 - Calling functions using function pointers

 - Passing function as an argument to another function

 - Callback functions

Exercise: - Registering For Multicast callbacks
Writing higher order functions to curry parameters

DAY5:

Sorting algorithms

- Numeric and string keys

- Stable sorts

- Internal and external sorts

- Bubble sort

- Insertion sort

- Selection sort

- Implement one of the above

Merge sort
Quick sort
Implement one of the above

Exercise: - Sorting of linked list
Insertion Sort with linked list

Storage classes, qualifiers and scope of identifiers

Automatic storage class
Static storage class
Register storage class
Block scope, Function scope and File (global) scope
Comparison with dynamic memory allocation
Constant and volatile qualifiers

Linkage

Symbol linkage across multiple source files
External linkage
Static linkage
A revisit of function prototypes and #include files

Trees

Generic trees
Binary trees
Mapping generic trees to Binary trees
Minimum and maximum heights of binary trees

Binary trees traversals
Preorder, inorder and post order
Level order traversal for binary heap implementation

Exercise: - Representing polished notation in Binary tree
Evaluating the expression tree

Binary search trees

Insertion, search and deletion operations
Implementation
Finding the height of a binary search tree
Implementation
Degenerate unbalanced binary search trees
Similarity to linked lists
Reference to AVL Trees and Red-Black Trees

ANNEXURE - II

C++ Complete with C++11 -14-17-20

1. OBJECTIVES

- ❖ To enable the participants to develop good object oriented design and use modern C++ effectively with parallelism

2. Duration

- ❖ This is a 25 **Days** Program.(200 hrs, 8 hrs per Day)

Entry Profile

- ❖ Medium degree of proficiency in “C”.
- ❖ Medium degree of proficiency in OO concepts

Day 1

Runtime memory layout of a application

- ❖ Code Segment
- ❖ Data Segment
- ❖ Heap
- ❖ Stack

Concepts of Object oriented

- ❖ The Object-oriented approach
- ❖ Abstraction, Encapsulation and Modularity
- ❖ Interface and implementation
- ❖ Fundamentals of Object Oriented Approach
- ❖ Why objects?
- ❖ Classes & Objects – the different perspectives
- ❖ State, Identity and Behavior
- ❖ Abstraction
- ❖ Encapsulation
- ❖ Inheritance & Class Hierarchy
- ❖ Polymorphism: Early v/s late binding
- ❖ Object relationships

Namespace

- ❖ What is a namespace?
- ❖ Using Namespace with Scope resolution
- ❖ Using Namespace with The using directive
- ❖ Using Namespace with The using declaration
- ❖ Namespace Aliases
- ❖ Namespace Composition

- ❖ Resolving Potential Clash

Day 2

Functions

- ❖ Inline function
- ❖ Function prototype
- ❖ Recursive Function
- ❖ Overloading Functions
- ❖ Pass by value v/s Pass by ref
- ❖ Default function Arguments
- ❖ Variable Parameter List
- ❖ Temporary objects

Class and Object concepts

- ❖ Classes and Objects
- ❖ Class declaration & object mechanism
- ❖ Objects & references
- ❖ Constructors and Destructors
- ❖ Overloaded constructors
- ❖ Copy constructor
- ❖ Destructors
- ❖ Managing the heap
- ❖ Objects & references
- ❖ Copy constructor
- ❖ Overloading copy constructor

Day 3

Data Members

- ❖ Memento Pattern

Function Pointers

- ❖ Callbacks using Function Pointer
- ❖ Synchronize v/s Asynchronous calls
- ❖ Observer Pattern

Container class

- ❖ Creating Container Class
- ❖ Iterator Pattern
- ❖ Composite Pattern
- ❖ Command Pattern

Object Attributes and Behaviour

- ❖ Static members and objects on the heap
- ❖ Static members of a class
- ❖ Static data members
- ❖ Static member functions
- ❖ Const data members
- ❖ Const member functions

Day 4

Friend Functions

- ❖ Friend and classes
- ❖ Object communication
- ❖ Friend functions
- ❖ Friend classes
- ❖ Const objects and const member functions
- ❖ Object composition & destruction mechanism

Operator Overloading

- ❖ Overloading unary & binary operators
- ❖ Overloading the input/output stream operators
- ❖ Operators that cannot be overloaded
- ❖ Conversion functions
- ❖ Returning objects & assignment operator
- ❖ Copy And Swap Idiom
- ❖ Overloading >> & << operators
- ❖ Cascading operators & returning reference

Day 5

Inference and Closures

- ❖ Type Inference
- ❖ Lambda Expressions

Dynamic Memory Management

Techniques for Memory Management

- ❖ Preventing Heap based objects
- ❖ Identifying object is on Heap or Stack
- ❖ Smart pointers

Day 6

Smart Pointers

- ❖ unique_ptr
- ❖ shared_ptr.
- ❖ weak_ptr.

Patterns for Memory Management

- ❖ Reference Counting
- ❖ Singleton pattern
- ❖ Creating Fly Weight objects

Object State Internals

- ❖ Application of static members
- ❖ Objects on the heap
- ❖ Use of this pointer
- ❖ Overloading of Constant members
- ❖ mutable data members
- ❖ Constant objects
- ❖ Overloading of static member functions

Day 7

Memory Management

Object Initialization

- ❖ Compiler Synthesized Constructor
- ❖ New vs malloc vs calloc
- ❖ Deep copy v/s Shallow copy
- ❖ Overloaded constructor
- ❖ Copy constructor & Generic Copy constructor
- ❖ Explicit constructor
- ❖ Copy Constructor v/s Assignment operator
- ❖ Initialization List
- ❖ Order of Initialization
- ❖ Initialization v/s Assignment

Day 8

Object Cleanup

- ❖ Destructor
- ❖ Delete vs Free
- ❖ Compiler Synthesized Destructor
- ❖ Preventing destroying object instance
- ❖ Avoiding dangling Pointers
- ❖ Destroying objects on the heap
- ❖ Destroying array of objects
- ❖ Issues with Dangling Pointer

Exception handling

- ❖ multiple catch blocks
- ❖ catch any block
- ❖ set_terminate functions
- ❖ custom exception class

Day 9

Exception Handling Issues

- ❖ Order of catch blocks
- ❖ Catching exception by value
- ❖ Throwing exception in constructor
- ❖ Throwing exception in destructor

Code Reusability Techniques using Inheritance & containment

- ❖ Containment vs Inheritance
- ❖ Private vs protected inheritance
- ❖ Hybrid Inheritance / virtual base class
- ❖ Changing scope of base member in derived class – The publicizer technique Inheritance
- ❖ The inheritance mechanism
- ❖ Types of inheritance
- ❖ Single level, multi level, multiple, hierarchical, hybrid
- ❖ Inheritance – is it allocation or accessibility
- ❖ The protected access specifier

Day 10

- ❖ Inheritance in private, protected & public mode
- ❖ Applying Inheritance, key abstraction and dynamic binding
- ❖ Constructors & destructors in a derived class
- ❖ Multiple inheritance
- ❖ Virtual Base class
- ❖ Invoking base class constructors
- ❖ Why a constructor is not inherited
- ❖ Is-a & Has-a relationship
- ❖ Nested classes & Containership
- ❖ Runtime Polymorphism, Virtual Functions & Dynamic Binding
- ❖ Pointers & Classes
- ❖ Pointers to Class Members

Day 11

Virtual functions

- ❖ Virtual member function
- ❖ Layering technique, issues related to tight-coupling and loose-coupling, avoiding transitive dependencies
- ❖ Pure virtual function
- ❖ Abstract class v/s Interface v/s Concrete class

Techniques using Inheritance & containment

- ❖ Runtime Inheritance using Decorator
- ❖ Changing Parent class using State
- ❖ Adopting to changes using Adopter
- ❖ Change implementation at Runtime using Strategy
- ❖ Reducing Complexity in Relationship using Mediator

Day 12

Virtual

Techniques using Virtual functions

- ❖ Proxy pattern using reinterpret_cast of interface pointers
- ❖ Virtual destructor
- ❖ Virtual constructor using Prototype pattern
- ❖ Dual dispatching

Advanced Techniques using Virtual functions

- ❖ Template Method
- ❖ Abstract Factory
- ❖ Adding Functionality horizontally using Visitor

Virtual function Issues

- ❖ Calling virtual function from constructor
- ❖ Calling virtual function from destructor
- ❖ Calling virtual function from non virtual member function
- ❖ Object Slicing
- ❖ Virtual destructor

Day 13

Virtual Internals

- ❖ Virtual functions in Single Inheritance
- ❖ Virtual functions in Multiple Inheritance
- ❖ Virtual Inheritance
- ❖ CRTP Idiom

RTTI and Casting Operators

- ❖ Introduction to RTTI
- ❖ Runtime Polymorphism & RTTI
- ❖ typeid & type_info
- ❖ The Casting operators
- ❖ const_cast
- ❖ static_cast
- ❖ reinterpret_cast
- ❖ dynamic_cast

Template Function and Classes

- ❖ Template Function
- ❖ Full Specialization
- ❖ Partial Specialization
- ❖ Class Template Full specialization
- ❖ Class Template Partial specialization
- ❖ Template Bloating

Day 14

Type Inference

- ❖ auto
- ❖ decltype
- ❖ Side effects
- ❖ Trailing return types

Lambda Expressions

- ❖ Why do we need this thing?
- ❖ Return type
- ❖ Lambda parameters
- ❖ Lambda body
- ❖ Storing lambdas
- ❖ std::function
- ❖ References to outside context
- Closures

Capturing in C++11

- ❖ Capturing by reference
- ❖ Default capture modes
- ❖ Capturing class members
- ❖ Limitations of capturing
- ❖ Mutable lambdas
- ❖ Conversion to function pointers, nested lambdas, recursion

Day 15

Template Features

- ❖ Variadic templates
- ❖ Working with parameter packs
- ❖ Traversing template parameter packs
- ❖ Constraining parameter packs to one type
- ❖ More places to expand a parameter pack
- ❖ Nested variadic templates
- ❖ One function template, two parameter packs

Template aliases

- ❖ Using using instead of typedef
- ❖ Closing angle brackets are officially allowed to tail-gate
- ❖ Local and unnamed types as template arguments

Class Features

- ❖ In-class initializers for non-static data members
- ❖ Delegating constructors
- ❖ Default methods
- ❖ Deleted methods
- ❖ override and final

Day 16

Move Semantics and Rvalue References

- ❖ Lvalue/rvalue revision
- ❖ const attribute .
- ❖ Reference initialization
- ❖ Rvalue references
- ❖ Move semantics implementation .
- ❖ Moving members .
- ❖ std::move

Perfect Forwarding

- ❖ The forwarding problem and solution .
- ❖ Reference collapsing and templates involving rvalue reference arguments .

Range-based for loop

nullptr

enum Changes

- ❖ Scoped enums
- ❖ Specifying the underlying type .
- ❖ Forward declaration

Day 17

Explicit Conversion Operators

Raw String Literals

The Dream of Uniform Initialization

- ❖ Embrace the braces .
- ❖ initializer_list .
- ❖ Narrowing conversions .
- ❖ Distortions of the uniformity continuum .
- ❖ Want a move-only type in your vector? .

Smart Pointers

- ❖ `unique_ptr`
- ❖ `shared_ptr`.
- ❖ `weak_ptr`.

Tuple Types

- ❖ `make_tuple`.
- ❖ Accessing tuple elements.
- ❖ Iterating over a tuple with template metaprogramming.
- ❖ Nested tuples.
- ❖ Tuple concatenation..

Day 18

Binding Arguments with `std::bind`

- ❖ Rearranging and duplicating parameters.
- ❖ Passing bound values by reference.
- ❖ Using `bind` with overloaded functions.
- ❖ Using `bind` with member functions.
- ❖ Binding data members.

Regular Expressions

- ❖ Wildcards
- ❖ Repetition.
- ❖ Character sets.
- ❖ Character classes.
- ❖ Word boundaries.
- ❖ Capture groups and back references.
- ❖ Escaping.

New STL Containers

- ❖ `forward_list`.
- ❖ `array`.
- ❖ Element operations.
- ❖ Container operations.
- ❖ Initialization.
- ❖ Array as tuple.
- ❖ Hash tables.

- ❖ unordered_map .
- ❖ Custom hash .
- ❖ unordered_multimap, unordered_set, unordered_multiset .

Concurrency Support: Synchronization

- ❖ Mutexes .
- ❖ Timed mutexes .
- ❖ Locking multiple mutexes .
- ❖ Locks .
- ❖ call_once .
- ❖ Condition variables .
- ❖ Limiting wait time .
- ❖ Other lockable types .
- ❖ notify_all_at_thread_exit .
- ❖ A note on const and mutable .

Day 19

Deprecated and Future Features

- ❖ C++14 .
- ❖ Return type deduction for functions .
- ❖ Generic lambdas .
- ❖ Extended capturing in lambdas .
- ❖ Revised restrictions on constexpr functions .
- ❖ constexpr variable templates .

Language Features:

- ❖ Structured Bindings
- ❖ if and switch with Initializers
- ❖ Inline Variables
- ❖ Aggregates with base classes
- ❖ Mandatory RVO, copy elision, and materialization
- ❖ Nested namespaces syntax
- ❖ static_assert() with one argument
- ❖ Lambda improvements (constexpr support, capturing *this)
- ❖ #_has_include
- ❖ New defined expression evaluation order
- ❖ Relaxed enumeration initialization
- ❖ Fixes for list initialization with auto

- ❖ noexcept as part of the function type
- ❖ new/delete with over-aligned data
- ❖ UTF-8 character literals
- ❖ Hexadecimal floating-point literals
- ❖ New attributes ([[nodiscard]], [[fallthrough]], and [[maybe_unused]])
- ❖ Attributes extensions (attributes for enumerations/namespaces and the using prefix)

Template Improvements:

- ❖ Class Template Argument Deduction
- ❖ Deduction Guides
- ❖ Compile-time if (constexpr if)
- ❖ Fold expressions
- ❖ Extending fold expressions
- ❖ Pack expansion in using declarations
- ❖ Non-type template parameters with placeholders (auto/decltype(auto))

Day 20

Basic Library Types:

- ❖ std::optional<>
- ❖ std::variant<>
- ❖ Polymorphism with std::variant<>
- ❖ std::any
- ❖ std::byte

Multi-Threading Features:

- ❖ shared_mutex
- ❖ scoped_lock
- ❖ is_always_lock_free()
- ❖ hardware interference sizes

C++20 Concepts:

- ❖ coroutines
- ❖ concepts
- ❖ designated initializers
- ❖ template syntax for lambdas
- ❖ range-based for loop with initializer
- ❖ likely and unlikely attributes
- ❖ deprecate implicit capture of this

- ❖ class types in non-type template parameters
- ❖ constexpr virtual functions
- ❖ explicit(bool)
- ❖ immediate functions
- ❖ using enum
- ❖ lambda capture of parameter pack
- ❖ char8_t

New library features:

- ❖ synchronized buffered outputstream
- ❖ std::span
- ❖ bit operations
- ❖ math constants
- ❖ std::is_constant_evaluated
- ❖ std::make_shared supports arrays
- ❖ starts_with and ends_with on strings
- ❖ check if associative container has element
- ❖ std::bit_cast
- ❖ std::midpoint
- ❖ std::to_array

Part 2 - Parallel Programming Using C++ (40 hrs) - Day 21-25

Prerequisite

- Template metaprogramming
- Lambda expressions

Parallel Programming with libraries (case studies/applications, numerical methods)

- Introduction of Parallel Computing
- Parallel Program Design
- Complexity, classification, SpeedUp, Efficiency
- Schedulers, Job Submission, Batch systems, Resource manager
- Pthreads
- OpenMP
- MPI
- Numerical methods

Case study with libraries

Fork-join parallelism

- Parallel Fibonacci
- Incrementing an array in parallel
- The sequential elision
- Executing fork-join algorithms

Race Conditions

- Synchronization Hardware
- ABA problem

Executing parallel algorithms

Experimenting with PASL

- Obtain source files
- Software Setup
- Starting with installed binaries
- Using the Makefile
- Task 1: Run the baseline Fibonacci
- Task 2: Run the sequential elision of Fibonacci
- Task 3: Run parallel Fibonacci
- Measuring performance with "speedup"
- Visualize processor utilization
- Strong versus weak scaling

Work efficiency

- Improving work efficiency with granularity control
- Determining the threshold

Automatic granularity control

- Complexity functions
- Controlled statements
- Controlled parallel-for loops

Simple Parallel Arrays

- Interface and cost model
- Allocation and deallocation
- Passing to and returning from functions
- Tabulation
- Higher-order granularity controllers
- Reduction
- Scan
- Derived operations

Parallel Sorting

- Quicksort
- Mergesort

Parallelism and Concurrency

- Race conditions and concurrency
- Eliminating race conditions by synchronization

Graphs

- Graph representation
- Breadth-first search

ANNEXURE - III

Python Programming Introduction to Python

- Overview
- Python Features
- Environment Setup
- Running Python
- IDLE
- Commercial and Free Python IDEs

Basic Syntax

- Interactive Mode
- Script Mode
- Python Identifiers
- Reserved Words
- Lines and Indentation
- Multi-Line Statements
- Quotations in Python
- Comments in Python
- Using Blank Lines
- User inputs
- Command Line Arguments

Variable Types

- Creating variables
- Assigning Values to Variables
- Accessing variables
- Multiple Assignment

Downloading, Installation & Configure Various Python Tools

- Python 2.x or 3.x on various OS platforms like Windows, Linux, Unix & MacOS
- Pycharm
- Anaconda & Spyder

Standard Data Types

- Numbers
- Strings
- Lists
- Tuples
- Dictionaries
- Sets

Python Data Types – Numbers

- int
- float
- complex
- Number type conversions
- Mathematical Functions
- Random Number Functions
- Trigonometric Functions
- Mathematical Constants

Python Data Types – Strings

- Accessing values in strings
- updating strings
- Escape Character
- String Special Operator
- String Formatting Operator
- Triple Quotes
- Unicode String
- Built-in String Methods

Python Data Types – Lists

- Creating the lists
- Accessing the values in lists
- Updating Lists
- Deleting List Elements
- Basic List Operators
- Indexing, Slicing and Matrixes
- Built-in List Functions & Methods

Python Data Types – Tuples

- Creating the tuple
- Accessing values in Tuples
- Updating Tuples
- Delete Tuples Elements
- Basic Tuples Operations
- Indexing, Slicing and Matrixes
- Built-in Tuple Functions

Python Data Types – Dictionary

- Creating a Dictionary
- Accessing Values in Dictionary
- Updating Dictionary
- Deleting Dictionary Elements
- Properties of Dictionary Keys
- Built-in Dictionary Functions & Methods

Python Data Types – Sets

- Creating A Set
- Accessing A Set
- Modifying A Set
- Removing Items From A Set
- Common Set Operations
- Built-in Set Functions & Methods

Python Operators

- List of Operators
- Arithmetic Operators
- Comparison Operators
- Assignment Operators
- Bitwise Operators
- Logical Operators
- Membership Operators
- Identity Operators
- Operators Precedence

Decision Making

- Single Statement Suites
- if statements
- if else statements
- if elif statements
- Nested statements

Python Loops

- Various Types of Loops
- while loop
- for loop
- nested loop
- Loop Control Statements
- break statement
- continue statement
- pass statement
- Iterator and Generator

Python Functions

- Defining a Function
- Calling a Function
- Pass by reference vs value
- Function Arguments
- Required Arguments
- Keyword Arguments
- Default Arguments
- Variable-length arguments
- The anonymous Functions
- The return Statement
- Scope of Variables
- Global vs Local Variables

Working with PIP

- Download PIP
- Configure Environment Variables
- Search Modules using PIP

- Download & Install Modules using PIP
- Various examples of PIP
- Downloading, Installing of Various modules

Object-Oriented Programming

- The class statement
- Creating a Class
- Methods
- The self
- “self” the self reference
- Object Methods
- Using Object Methods
- The__init__method (The default constructor)
- Using the__init__method Private(_var) & Semi-private variables(_var)
- Class and Object Variables
- Using Class and Object Variables

Inheritance

- Using Inheritance
- Multiple-inheritance
- Operator Overloading
- operator — Standard operators as functions

collections.abc — Abstract Base Classes for Containers

- Errors and Exceptions
- User-defined Exceptions
- Defining Clean-up Actions
- Predefined Clean-up Actions

Functional Programming

- itertools — Functions creating iterators for efficient looping
- functools — Higher-order functions and operations on callable objects
- operator — Standard operators as functions
- Decorators
- List Comprehensions - Nested List Comprehensions
- Looping Techniques
- More on Conditions
- Comparing Sequences and Other Types
- Iterators
- Generators

The import statement

- The from import statement
- The from import * statement
- Executing modules as scripts
- Locating Modules

Reading and Writing Files

- The write() method
- The read() method
- File Positions
- Renaming and Deleting Files
- The rename() Method & remove() method

Directories in Python

- The mkdir() method
- The chdir() method
- The getcwd() method
- The rmdir() method
- File & Directory Related Methods

Error & Exceptions Handling in Python

- Assertions in Python
- The assert statement
- What is an exception?
- The except clause with no exceptions
- The except clause with multiple exceptions
- The try-finally clause
- Argument of an exception
- Raising an exception
- User-defined exceptions

Working with the Databases

- What is MySQL?
- Database Connection
- Creating Database Table
- Insert operation
- Read operation
- Update operation
- Delete operation
- Performing Transactions
- Commit Operation
- Rollback Operation
- Error Handling

Handling HTTP & HTTPS

- Introduction to HTTP Protocol
- Different HTTP Method GET, POST, PUT
- HTTP Requests
- HTTP Response
- HTTP Headers
- Custom HTTP Requests
- Request Status Codes
- HTTP Authentication
- HTTP Data Download
- Implement using requests library

Python GUI – PyQT and Tkinter

- Installing Python 3 and Tcl/Tk, PyQt for Mac or Windows
- Creating and configuring themed Tk widgets
- Decorating the GUI with text labels and images
- Capturing input from buttons, menus, and entry fields
- Presenting choices with check boxes and radio buttons
- Using geometry managers to lay out the GUI
- Organizing widgets inside of frames and windows
- Handling user actions with event-driven programming

Flask

- Introduction To Flask For Beginners
- Flask Template, Form, View, And Redirect

- Flask Database Handling
- Flask App And Flask Project Layout With Blueprint & Bootstrap
- Flask Design Patterns And Best Practices For Web Applications
- Flask API
- Extending Flask With APIs

Mathematical Computation by numpy

- NumPy Overview
- Properties, Purpose, and Types of ndarray
- Class and Attributes of ndarray Object
- Basic Operations: Concept and Examples
- Accessing Array Elements: Indexing, Slicing, Iteration, Indexing with Boolean Arrays
- Copy and Views
- Universal Functions
- Shape Manipulation & Broadcasting
- Linear Algebra using numpy
- Stacking and resizing the array

Data Analysis with Pandas

- Introduction to Pandas
- Data Structures
- Series & DataFrame
- DataFrame basic properties
- Importing excel sheets, csv files, loading data from html
- Importing and exporting json files
- Selection of columns
- Filtering Dataframes
- Descriptive Analysis with pandas
- Data Cleaning Handling Missing Values
- Handling unwanted columns
- Handling outliers
- Handling duplicated entries
- Finding unique values
- Creating new categorical features from continuous variable
- Grouby operations
- Grouby statistical Analysis
- Apply method

Data Visualization

- Introduction to Data Visualization
- Python Libraries
- Data Visualization Best practices
- Matplotlib Features
- Line Properties Plot with (x, y)
- Controlling Line Patterns and Colors
- Set Axis, Labels, and Legend Properties
- Alpha and Annotation
- Multiple Plots
- Subplots
- Scatterplots
- Pie Charts
- barplots
- Types of Plots and Seaborn
- Boxplots
- Distribution Plots

- Heatmaps
- Swarmplots and countplots
- Pointplots

Case Study – using NumPy, Pandas, matplotlib and seaborn – EDA

- Customer Churn Dataset
- Insurance Dataset
- Drug Dataset
- CCPP Dataset

How to design Analytic Report

Exploratory Analytics with Data Visualization and Statistics

- Univariate Analytics
- Bivariate Analytics
- Multivariate Analytics

Regular Expression

- Regular Expression Syntax
- Example of w+ and ^ Expression
- Example of \s expression in re.split function
- Using regular expression methods
- Using re.match()
- Finding Pattern in Text (re.search())
- Using re.findall for text
- Python Flags
- Example of re.M or Multiline Flags

Advanced Python (Part-2 Prerequisite for Parallel Computing)

Data Serialization

- Serialization using JSON
- Serialization using Pickle

Multiprocessing

- Running Two Simple Processes
- Using Pool and Map

Multithreading

- Basics of multithreading
- Communicating between threads
- Creating a worker pool
- Advanced use of multithreads
- Stoppable Thread with a while Loop

Processes and Threads

- Global Interpreter Lock
- Running in Multiple Threads
- Running in Multiple Processes
- Sharing State Between Threads
- Sharing State Between Processes

Getting Started with Parallel Computing and Python

The parallel computing memory architecture

- SISD
- MISD
- SIMD
- MIMD

Memory organization

- Shared memory
- Distributed memory

- Massively parallel processing
- A cluster of workstations
- The heterogeneous architecture

Parallel programming models

- The shared memory model
- The multithread model
- The message passing model
- The data parallel model

How to design a parallel program

- Task decomposition
- Task assignment
- Agglomeration
- Mapping
 - Dynamic mapping
 - Manager/worker
 - Hierarchical manager/worker
 - Decentralize

How to evaluate the performance of a parallel program

- Speedup
- Efficiency
- Scaling
- Amdahl's law
- Gustafson's law

Python in a parallel world

Introducing processes and threads

Start working with processes in Python

Start working with threads in Python

Thread-based Parallelism

- Using the Python threading module
- Define a thread
- Determine the current thread
- Use a thread in a subclass
- Thread synchronization with Lock and RLock
- Thread synchronization with RLock
- Thread synchronization with semaphores
- Thread synchronization with a condition
- Thread synchronization with an event
- Using the with statement
- Thread communication using a queue
- Evaluating the performance of multithread applications

Process-based Parallelism

- Spawn a process
- Name a process
- Run a process in the background
- Kill a process
- Use a process in a subclass
- Exchange objects between processes
 - Using queue to exchange objects
 - Using pipes to exchange objects
- Synchronize processes
- Manage a state between processes
- Use a process pool
- Using the mpi4py Python module
- Point-to-point communication
- Avoiding deadlock problems

- Collective communication using broadcast
- Collective communication using scatter
- Collective communication using gather
- Collective communication using Alltoall
- The reduction operation
- How to optimize communication

Asynchronous Programming

- Using the concurrent.futures Python modules
- Dealing with the process and thread pool
- Event loop management with Asyncio
- Handling coroutines with Asyncio
- Task manipulation with Asyncio
- Dealing with Asyncio and Futures

Distributed Python

- Using Celery to distribute tasks
- Create a task with Celery
- Scientific computing with SCOOP
- Handling map functions with SCOOP
- Remote Method Invocation with Pyro4
- Chaining objects with Pyro4
- Developing a client-server application with Pyro4
- Communicating sequential processes with PyCSP
- Using MapReduce with Disco
- Remote procedure call with RPyC

GPU Programming with Python

- Using the PyCUDA module
- A hybrid programming model
- The kernel and thread hierarchy
- Build a PyCUDA application
- Understanding the PyCUDA memory model with matrix manipulation
- Kernel invocations with GPUArray
- Evaluating element-wise expressions with PyCUDA
- The MapReduce operation with PyCUDA
- GPU programming with NumbaPro
- Using GPU-accelerated libraries with NumbaPro
- Using the PyOpenCL module
- Build a PyOpenCL application
- Evaluating element-wise expressions with PyOpenCL
- Testing your GPU application with PyOpenCL